



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

Prompt Engineering for C and C++ 2026-2027

Course: B.Tech CSE (Core)

Registration number: 25BCON1594

Section: I

Semester: III

Submitted to: Mr. Ajay Sharma

Submitted by: Chehak Gupta

Practical 4: Google Colab and Code Conversion

Sign up done, new notebook created and ran multiple codes!



```
[4] print("Hello from Colab")
✓ Os
... Hello from Colab

[5] n = int(input("Enter a number"))
    fact = 1
    if(n==0):
        print("1")
    else:
        for i in range(n,0,-1):
            fact=fact*i
        print(fact)
✓ 8s
... Enter a number5
    120

[7] d1 =dict()
    d1 ={"Name": "Dhruv", "Roll":1607, "Marks":94}
    print(d1)
✓ Os
... {'Name': 'Dhruv', 'Roll': 1607, 'Marks': 94}
```

Introduction

This assignment focuses on **Prompt Engineering and the use of Large Language Models (LLMs) for code conversion**. A C program was provided to an LLM with a prompt asking it to convert the code into Python while maintaining the same functionality. The original C code and the Python code generated by the LLM were then compared to understand how the LLM interprets programming concepts and translates them between different languages. The comparison focuses on changes in **variable declaration, loop syntax, output statements, and variable updates**, along with a reflection on how effective prompting can influence the quality and structure of the generated code.

1. C Program for Factorial of a number n

```
1  #include <stdio.h>
2
3  int main() {
4      // your code goes here
5      int n = 5; int fact = 1;
6      for (int i = 1; i <= n; i++) {
7          fact = fact * i;
8      }
9      printf("Factorial of %d = %d", n, fact);
10 }
11
12
```

Output:

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.676 Mb

Your Output

Factorial of 5 = 120

My converted code in Python

```
[5] n = int(input("Enter a number"))
✓ 8s fact = 1
if(n==0):
    print("1")
else:
    for i in range(n,0,-1):
        fact=fact*i
    print(fact)

Enter a number5
120
```

Asking LLM to check the converted code

I have written this Python code myself after translating it from C. Do not rewrite it. Tell me what a Python programmer would do differently, and explain why each change is an improvement.

Response given by LLM

Your logic is completely sound, and translating from C is a great way to learn Python's mechanics. Here is how an experienced Python programmer would adjust your approach to make it more idiomatic (often called "Pythonic"):

- **Drop the parentheses in conditions:** You wrote `if(n==0):`, which is a leftover habit from C. Python does not require parentheses around `if` or `while` conditions. Writing `if n == 0:` is the standard style and makes the code less cluttered.
- **Use augmented assignment:** Instead of writing `fact = fact * i`, Python developers use `fact *= i`. It achieves the exact same result but is more concise and saves you from repeating the variable name.
- **Eliminate the `if/else` by looping upwards:** You can actually drop the `if(n==0)` check entirely. If you set `fact = 1` and change your loop to count up (`for i in range(1, n + 1):`), an input of `0` creates an empty range (`range(1, 1)`). The loop simply bypasses itself, and the script correctly prints the initial `1`. This flattens the logic and removes unnecessary branching.
- **Format with PEP 8 spacing:** Python has an official style guide called PEP 8, which focuses heavily on readability. A Python programmer would add spaces around assignments and operators (e.g., `fact = 1` instead of `fact =1`). Additionally, adding a colon and a space to your input prompt (`"Enter a number: "`) prevents the user's typing from awkwardly colliding with the prompt text.
- **Leverage the Standard Library:** In the real world, a Python programmer wouldn't write this loop at all. They would `import math` at the top of the file and use `math.factorial(n)`. Python is famously a "batteries included" language, and its built-in math functions are written in C under the hood, making them exponentially faster and more rigorously tested for edge cases than manual loops.

2. C Program to print Fibonacci Series

```
1  #include <stdio.h>
2
3  int main() {
4      // your code goes here
5      int n = 10, a=0, b=1, next;
6      for (int i=0; i<n; i++) {
7          printf("%d ", a);
8          next = a + b;
9          a = b;
10         b = next;
11     }
12     return 0;
13 }
14
15
16 |
```

Output:

Status : Successfully executed

Time:
0.0000 secs

Memory:
1.66 Mb

Your Output

0 1 1 2 3 5 8 13 21 34

Prompt to convert it

```
Consider yourself a python programmer, translate this C
code of printing Fibonacci series to python code.

#include <stdio.h>

int main() {
    // your code goes here
    int n = 10, a=0, b=1, next;
    for (int i=0; i<n; i++) {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }
    return 0;
}
```

Converted code in Python by LLM

Here is the direct Python translation of your C code.

```
Python

n = 10
a, b = 0, 1

for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Key Python Differences

- **Parallel assignment:** Python allows you to update multiple variables simultaneously (`a, b = b, a + b`). This evaluates the right side completely before assigning it to the left side, eliminating the need for your temporary `next` variable.
- **Print formatting:** By default, Python's `print()` adds a newline after every call. Adding `end=" "` replicates your `printf("%d ", a)` behavior by appending a space instead of dropping to a new line.
- **Loop variable:** Using `_` (underscore) instead of `i` is a standard Python convention when the index variable itself isn't actually used inside the loop body.

👍 🔄 ↻ 📄 ...

3. C Program of printing Name, ID and Marks of a student using Struct

```
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Student {
5      char name[50];
6      int roll;
7      float marks;
8  };
9
10 int main() {
11     struct Student s1;
12     strcpy(s1.name, "Dhruv");
13     s1.roll = 1607;
14     s1.marks = 94;
15
16     printf("Name: %s\n", s1.name);
17     printf("Roll: %d\n", s1.roll);
18     printf("Marks: %.1f\n", s1.marks);
19
20     return 0;
21 }
```

Output:

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.796 Mb

Your Output

Name: Dhruv
Roll: 1607
Marks: 94.0

My converted code in Python

```
[7] d1 = dict()
✓ Os d1 = {"Name": "Dhruv", "Roll": 1607, "Marks": 94}
print(d1)
... {'Name': 'Dhruv', 'Roll': 1607, 'Marks': 94}
```

Prompt to convert it by LLM

Consider yourself a python programmer, translate this C code into python code.

```
#include <stdio.h>
#include <string.h>

struct Student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    struct Student s1;
    strcpy(s1.name, "Rahul");
    s1.roll = 101;
    s1.marks = 87.5;

    printf("Name: %s\n", s1.name);
    printf("Roll: %d\n", s1.roll);
    printf("Marks: %.1f\n", s1.marks);

    return 0;
}
```

Converted code in Python by LLM

Here are two ways to translate this C struct into Python.

The most modern and Pythonic approach is using a `dataclass`, which perfectly mirrors the behavior of a C struct while keeping the code clean.

Method 1: Using a Dataclass (Recommended)

Python

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    roll: int
    marks: float

# Create an instance and assign values
s1 = Student(name="Rahul", roll=101, marks=87.5)

# Print the values
print(f"Name: {s1.name}")
print(f"Roll: {s1.roll}")
print(f"Marks: {s1.marks:.1f}")
```


Method 2: Using a Standard Python Class

If you don't want to import any modules, you can use a traditional class with an `__init__` constructor method:

Python



```
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

# Create an instance and assign values
s1 = Student("Rahul", 101, 87.5)

# Print the values
print(f"Name: {s1.name}")
print(f"Roll: {s1.roll}")
print(f"Marks: {s1.marks:.1f}")
```

Key differences to note:

- Python doesn't require pre-defining array sizes for strings (like `char name[50]`). Python strings dynamically adjust to the text length.
- We use Python's f-strings (`f"..."`) for printing, which are safer and more readable than C's `printf` format specifiers.

Difference Log

What I changed	How C did it	How Python did it
Variable Declaration	Variables need a data type, e.g. <code>int n = 10, a = 0, b = 1, next;</code>	No type declaration is needed. Python infers the type: <code>n = 10, a, b = 0, 1</code>
Loop syntax	Uses a C-style for loop: <code>for (int i = 0; i < n; i++)</code>	Uses <code>range(n)</code> to repeat the loop: <code>for _ in range(n)</code>
Printing output	Uses <code>printf("%d ", a);</code> and a format specifier <code>%d</code> for integers.	Uses <code>print(a, end=" ");</code> <code>end=" "</code> prevents moving to a new line after each number.
Grouping data (struct)	C does not group <code>a</code> and <code>b</code> together here; they are separate integer variables. Updating them requires a temporary variable next.	Python can update both variables together using multiple assignment : <code>a, b = b, a + b</code> . This is not a struct; it is tuple-based multiple assignment.
One thing that surprised me	C needs an extra variable next to safely update <code>a</code> and <code>b</code> because assignments happen separately.	Python can update <code>a</code> and <code>b</code> simultaneously in one line: <code>a, b = b, a + b</code> , making the Fibonacci logic much shorter.

Reflection: C vs Python

Translating the C code into Python showed me that Python can achieve the same result with much less code. C requires explicit data types, a more detailed loop structure, and a temporary variable to update the Fibonacci values. Python is more concise because it handles variable types automatically and supports multiple assignment. I found Python easier to read, while C gives more direct control over how variables and operations are handled.